Machine Learning

K-Nearest Neighbours Regression

Tarapong Sreenuch

8 February 2024

克明峻德，格物致知

We've been in our flat for decades and now need more room for family.

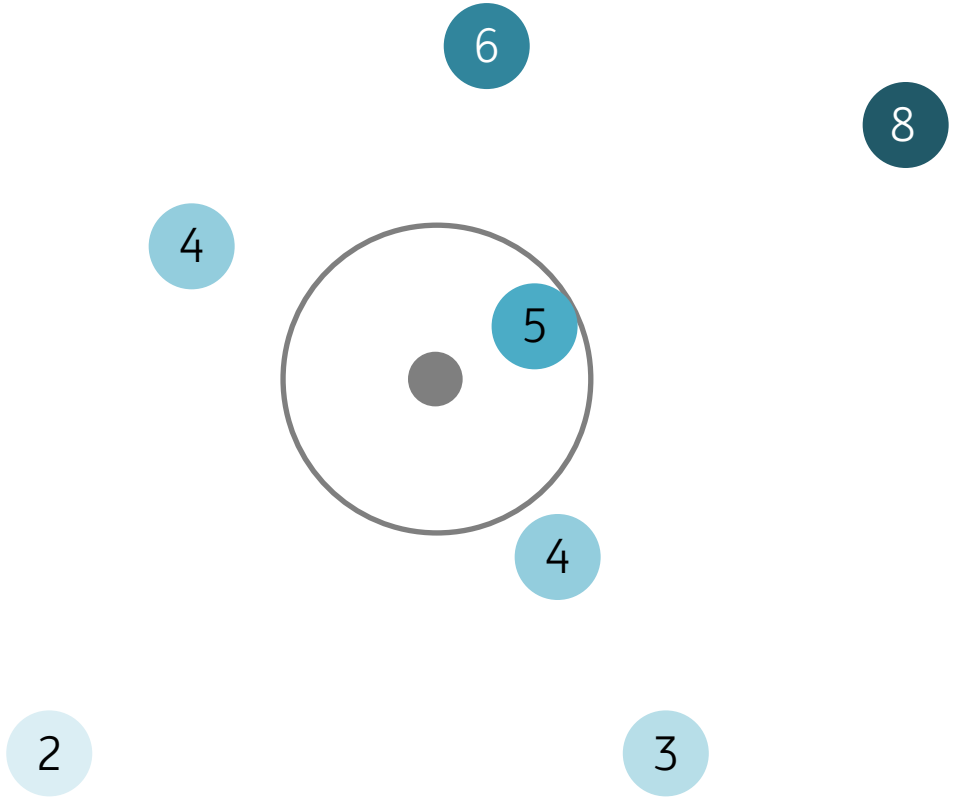
Our neighborhood has recently seen several flats go up for sale.

Q: How much should we list our flat (or unit) for?

A: By referring to the sold price of the flats (or units) which are similar, e.g. block, floor, flat type, to one we have.

K-Nearest Neighbours

The KNN algorithm predicts the target value for a test dataset by averaging the labels of its nearest neighbors in the training dataset's feature space. The 'K', representing the number of neighbors considered, is a key hyperparameter that can be adjusted to optimize model performance. Typically, the prediction is the mean or sometimes the median of these neighbors' target values, depending on the application's needs.



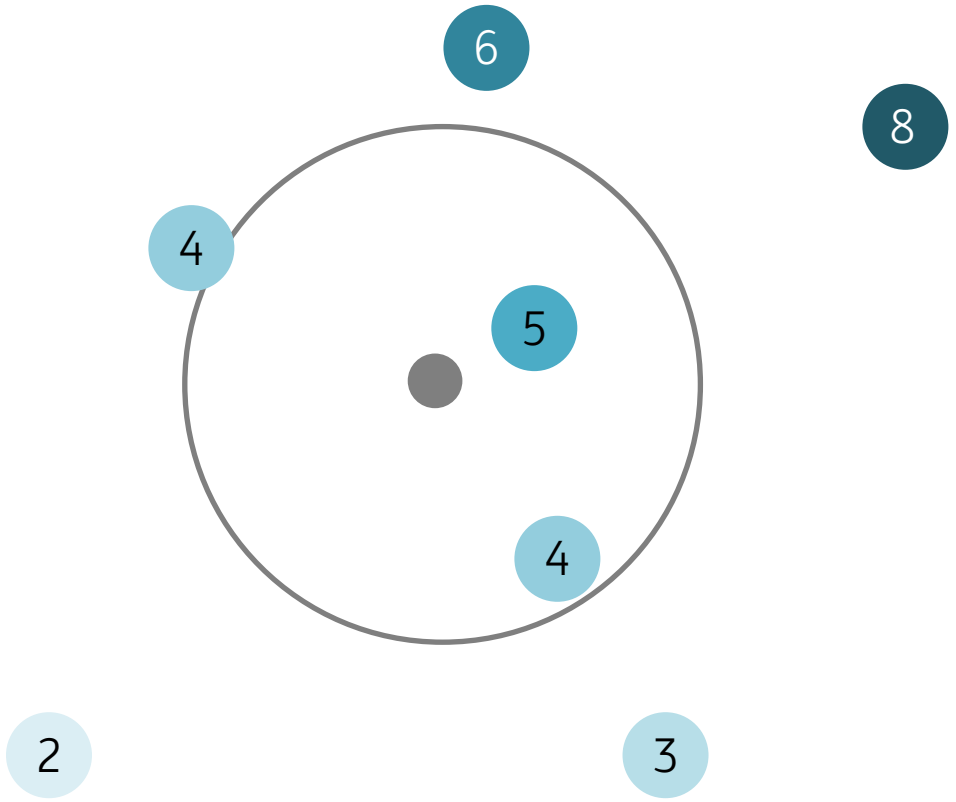
K = 1

NN's Labels: [5]

Prediction: 5.00

K-Nearest Neighbours

The KNN algorithm predicts the target value for a test dataset by averaging the labels of its nearest neighbors in the training dataset's feature space. The 'K', representing the number of neighbors considered, is a key hyperparameter that can be adjusted to optimize model performance. Typically, the prediction is the mean or sometimes the median of these neighbors' target values, depending on the application's needs.



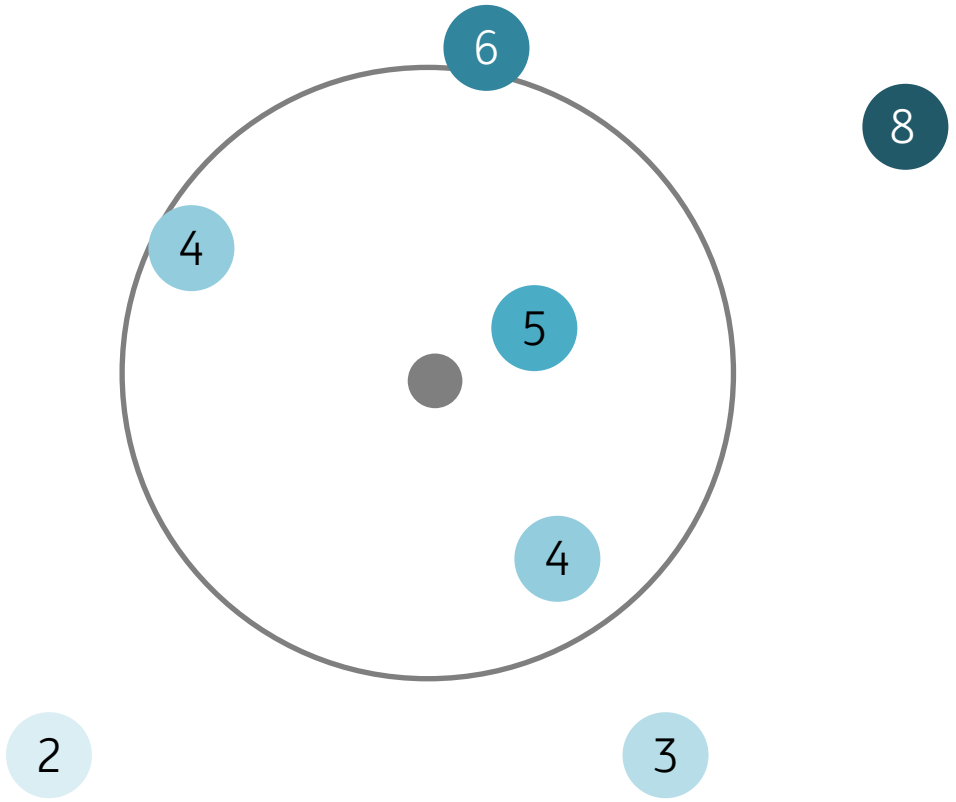
K = 2

NN's Labels: [5, 4]

Prediction: **4.5**

K-Nearest Neighbours

The KNN algorithm predicts the target value for a test dataset by averaging the labels of its nearest neighbors in the training dataset's feature space. The 'K', representing the number of neighbors considered, is a key hyperparameter that can be adjusted to optimize model performance. Typically, the prediction is the mean or sometimes the median of these neighbors' target values, depending on the application's needs.



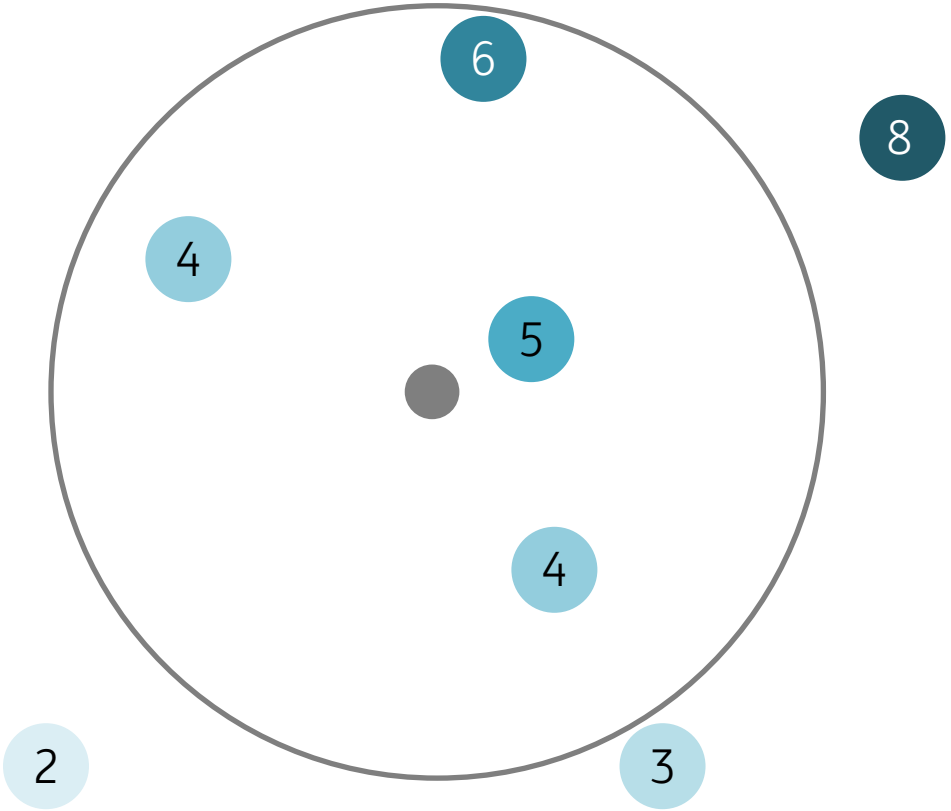
K = 3

NN's Labels: [5, 4, 4]

Prediction: **4.33**

K-Nearest Neighbours

The KNN algorithm predicts the target value for a test dataset by averaging the labels of its nearest neighbors in the training dataset's feature space. The 'K', representing the number of neighbors considered, is a key hyperparameter that can be adjusted to optimize model performance. Typically, the prediction is the mean or sometimes the median of these neighbors' target values, depending on the application's needs.



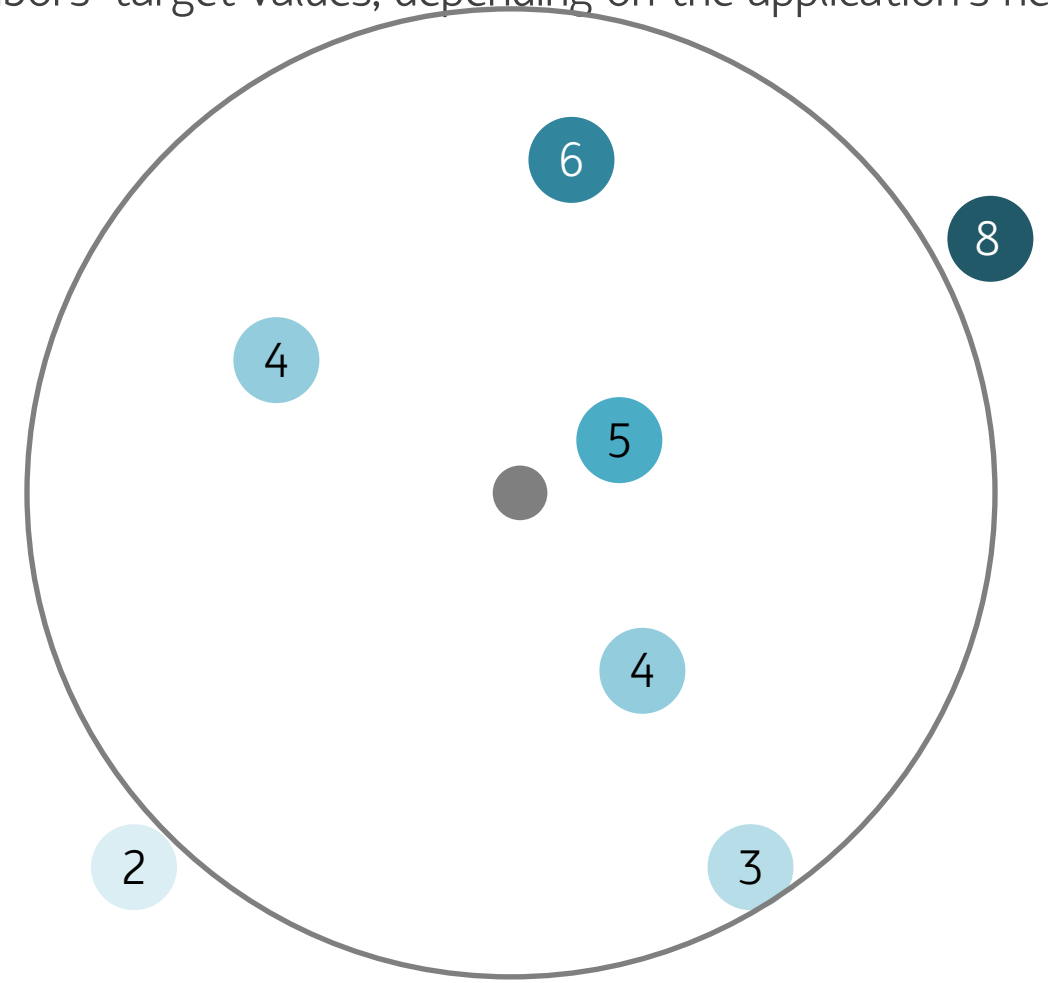
K = 4

NN's Labels: [5, 4, 4, 6]

Prediction: **4.75**

K-Nearest Neighbours

The KNN algorithm predicts the target value for a test dataset by averaging the labels of its nearest neighbors in the training dataset's feature space. Typically, the prediction is the mean or sometimes the median of these neighbors' target values, depending on the application's needs.



K = 5

NN's Labels: [5, 4, 4, 6, 3]

Prediction: **4.40**

Pseudocode for KNN Regressor

```
Function kNN_Regress(new_point, data_points, labels, k)
    Begin
        // Step 1: Calculate distances
        Initialize an empty list: distances
        For each point in data_points do
            Calculate the distance between new_point and point
            Add the distance and the corresponding label to the distances list

        // Step 2: Sort distances
        Sort distances in ascending order (based on distance values)

        // Step 3: Select k nearest neighbors
        Select the first k elements from the sorted distances list

        // Step 4: Compute average of labels
        Initialize sum: label_sum = 0
        For each neighbor in the k nearest neighbors do
            Add neighbor's label to label_sum

        // Step 5: Determine the predicted value
        predicted_value = label_sum / k

        Return predicted_value
    End
```

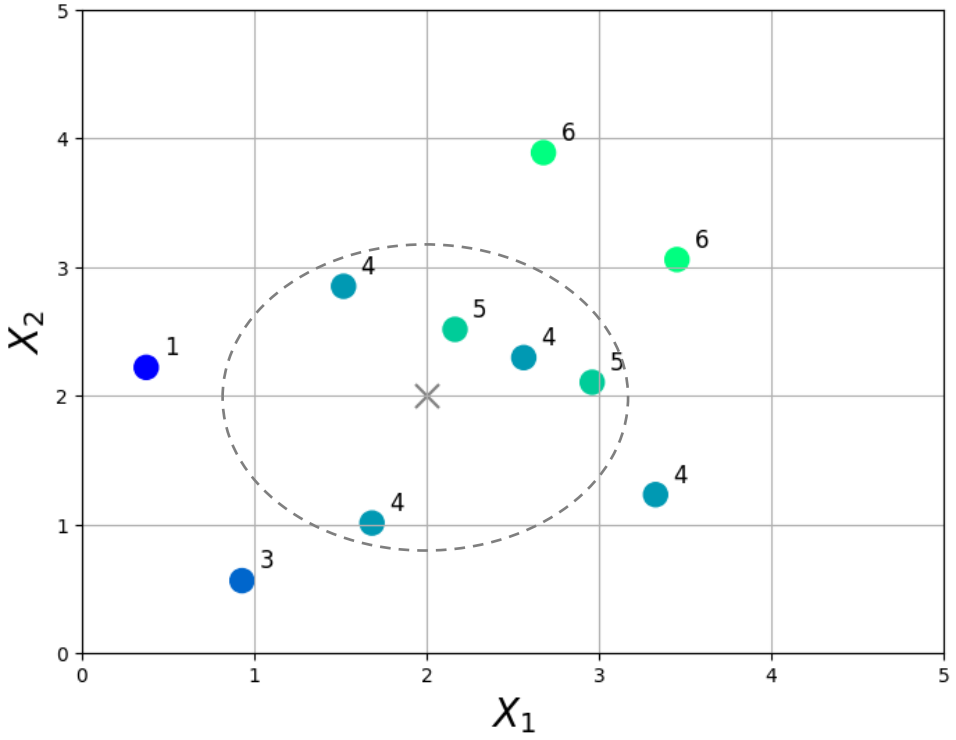
Python Code Snippet

```
import numpy as np
from sklearn.metrics import pairwise_distances

def kNN_Regress(new_point, data_points, labels, k, distance_metric='euclidean'):
    # Step 1: Calculate distances using pairwise_distances
    distances = pairwise_distances(data_points, [new_point], metric=distance_metric).ravel()

    # Step 2 & 3: Sort distances and select k nearest neighbors
    k_nearest_neighbors_indices = np.argsort(distances)[:k]
    k_nearest_labels = labels[k_nearest_neighbors_indices]

    # Step 4: Compute average of labels
    predicted_value = np.mean(k_nearest_labels)

    # Return the predicted value
    return predicted_value
```

Usage Example

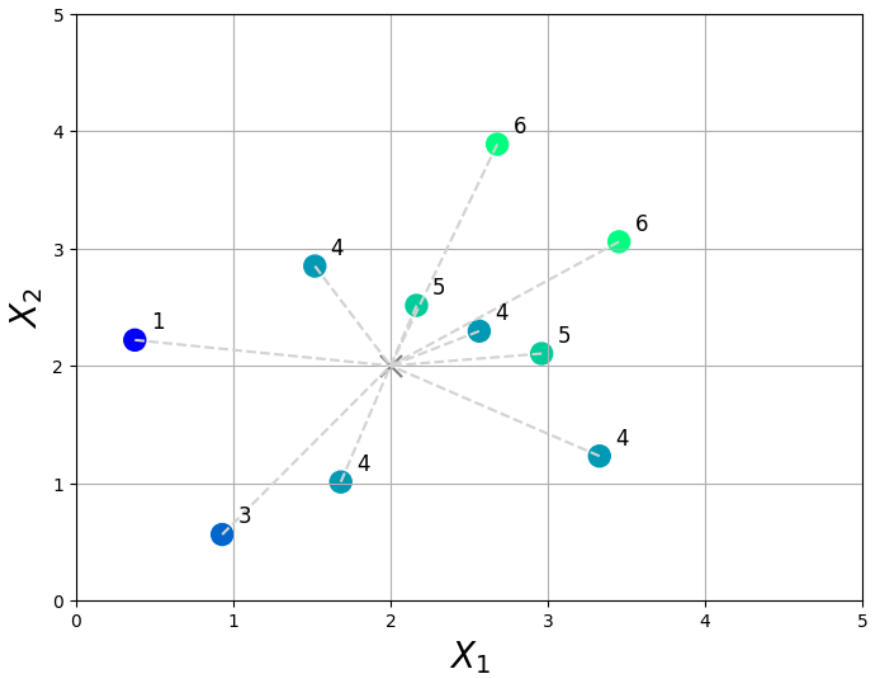
```
# Example usage
k = 5
distance_metric = 'euclidean'  # Can be 'euclidean', 'manhattan', 'cosine', etc.

# Predict the target value using regression
predicted_value = kNN_Regress(new_point, data_points, labels, k, distance_metric)
print("Predicted Value:", predicted_value)
```

```
Predicted Value: 4.40
```

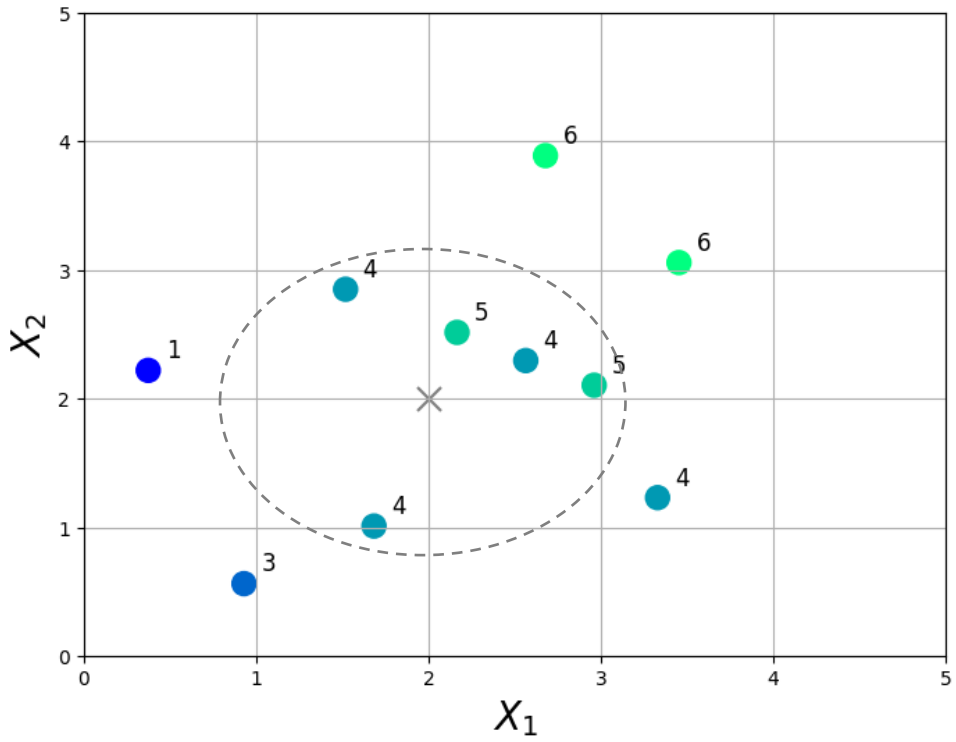


Step 1: Pairwise Distance Calculation

```
# Step 1: Calculate distances using pairwise_distances
distances = pairwise_distances(data_points, [new_point], metric=distance_metric).ravel()
```



Step 2 & 3: Finding k Nearest Neighbours

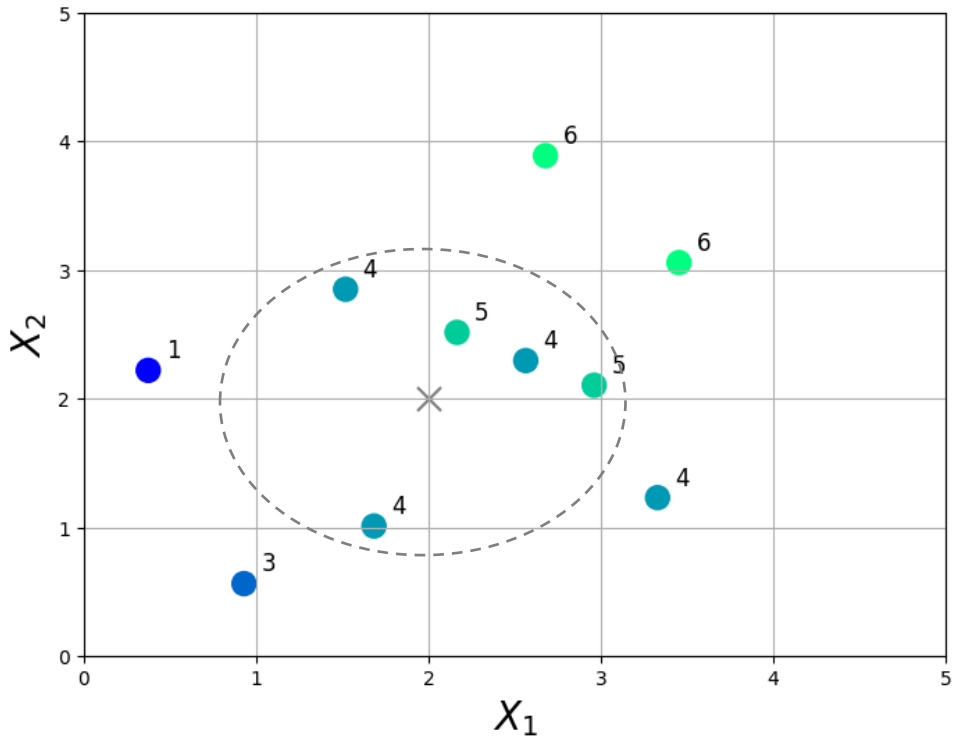



```
# Step 2 & 3: Sort distances and select k nearest neighbors
k_nearest_neighbors_indices = np.argsort(distances)[:k]
k_nearest_labels = labels[k_nearest_neighbors_indices]
```

NN's Labels: [5, 4, 5, 4, 4]

K = 5

Step 4 & 5: Prediction based on Majority Voting



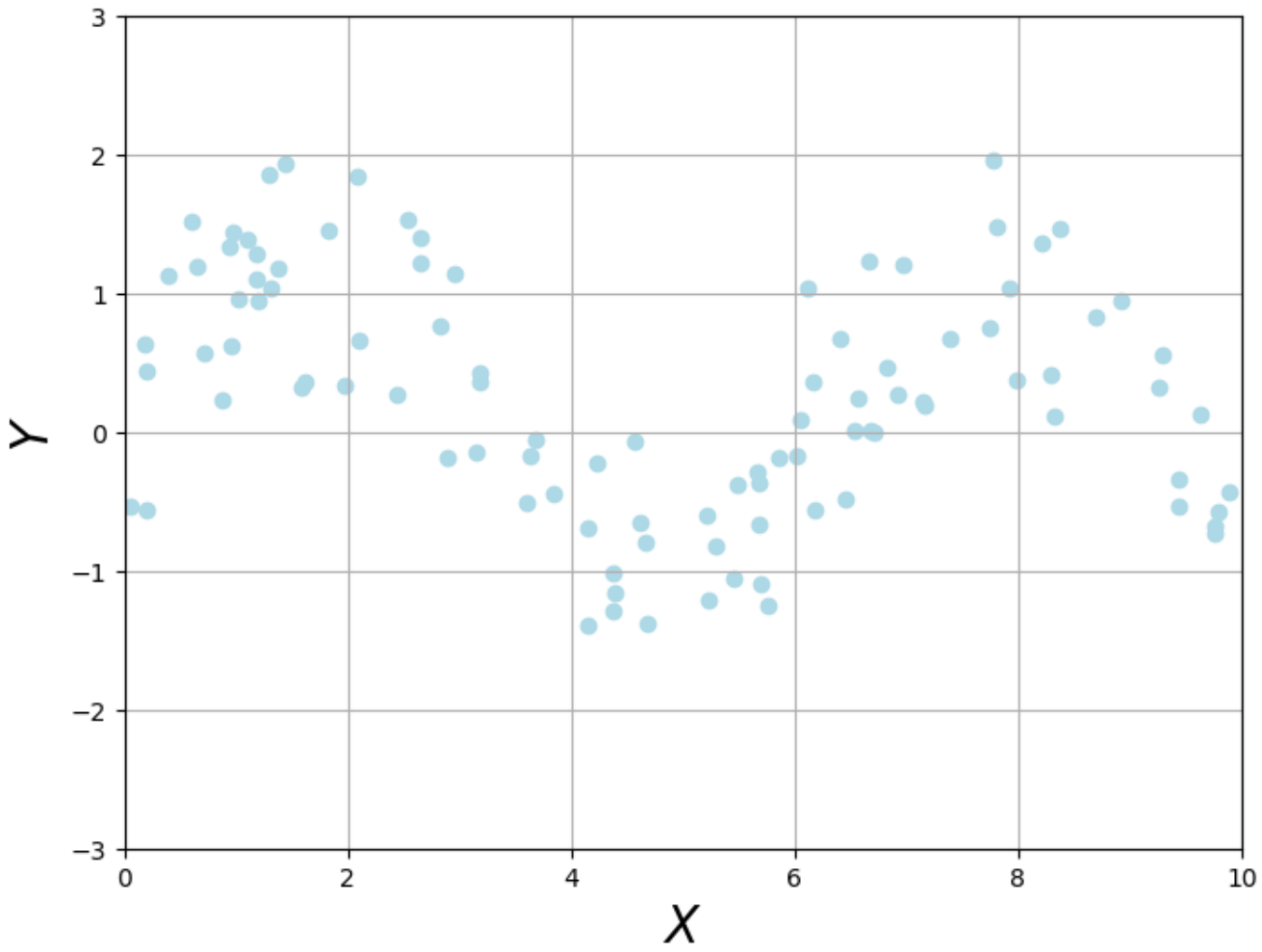

```
# Step 4: Compute average of labels
predicted_value = np.mean(k_nearest_labels)
```

K = 5

NN's Labels: [5, 4, 5, 4, 4]

Prediction: 4.40

Example Dataset



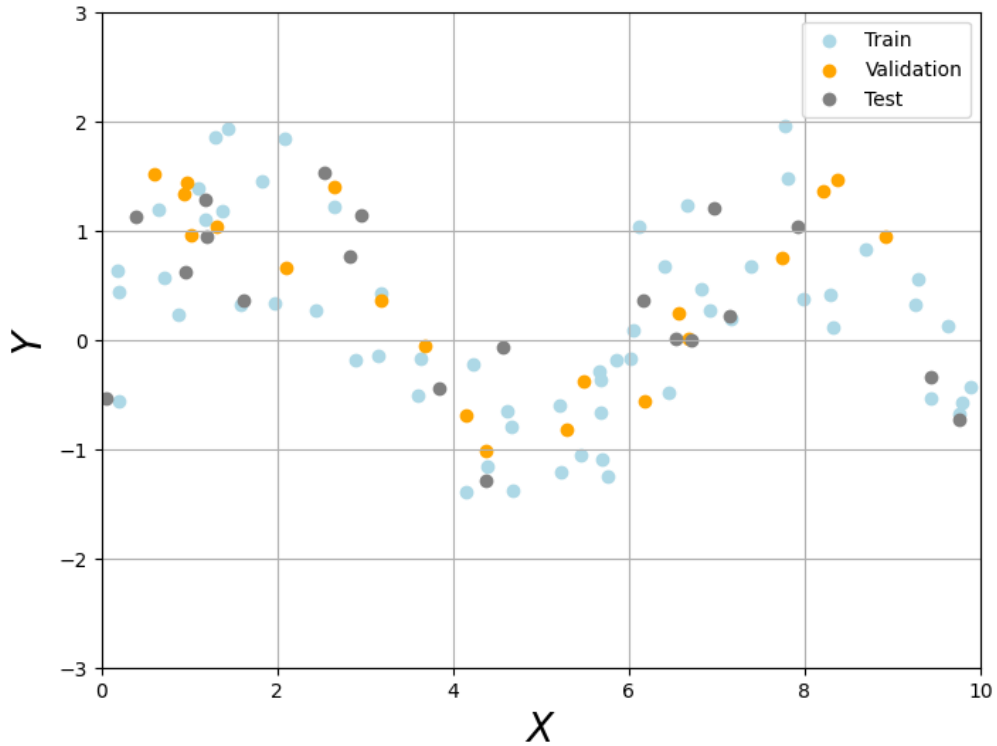
What to Expect from Our ML Model?

- In trading, we learn from the historical data and come up with a trading algorithm.
- To deploy or not, we need to perform backtesting on different equities, which have never seen before.
- We can also segment the data according to time periods, i.e. we learn from the past and predict the future.

- Like wise, we prefer an ML model which is performs well and more importantly consistent on train, validation and test datasets. Though the data is unseen, we want to be able to anticipate of what is going to happen.
- The model must generalise well and learn underlying relationships in the data.

Train, Validation and Test Datasets

```
from sklearn.model_selection import train_test_split

# First, split the data into train (60%) and temp (40%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=42)

# Then split the temp data into validation (50% of 40% ⇒ 20% of total) and test (remaining 50% of 40% ⇒ 20% of total)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```



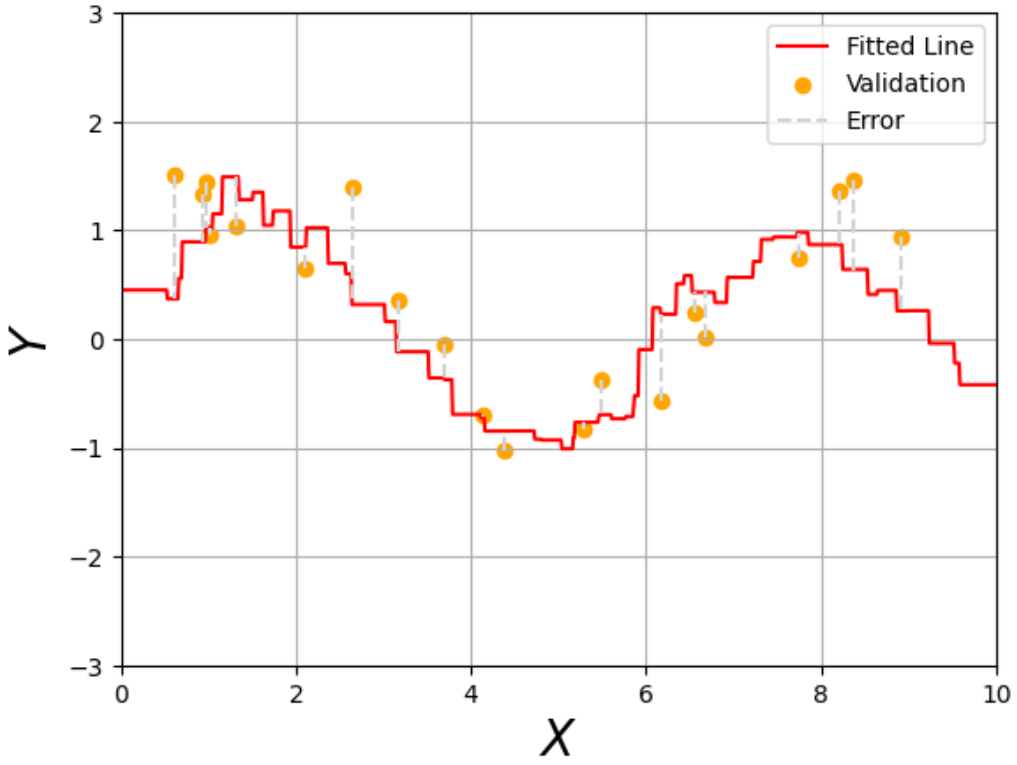
- (Train+Validation):Test is typicaly either 0.8:0.2 or 0.7:0.3.
- **Test** dataset is a proxy of unseen data, and it will only be used in the final evaluation.
- We train our ML model on the **Train** dataset.
- **Validation** datest is used to fine-tune or optimise the ML model. Here, we find a set of hyper-parameters, e.g. k in the kNN context, that will result in the best performance on the Validation dataset.

scikit-learn: KNeighborsRegressor

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

# Define k and the distance metric
k = 5
distance_metric = 'euclidean'

# Create an instance of KNeighborsRegressor
knn = KNeighborsRegressor(n_neighbors=k, metric=distance_metric)

# Fit the model on the training data
knn.fit(X_train, y_train)

# Predict the target values on the validation set
predictions = knn.predict(X_val)

# Calculate the mean squared error of the predictions
mse = mean_squared_error(y_val, predictions)
print("Mean Squared Error of the k-NN regressor on the validation set:
{:.2f}".format(mse))
```

```
Mean Squared Error of the k-NN regressor on the validation set: 0.29
```

Regression Line



$$\text{Error (Residue)} \quad = \quad y - \hat{y}$$

where y and $\hat{y}$ are the observed and predicted values, respectively.

Mean Squared Error (MSE):

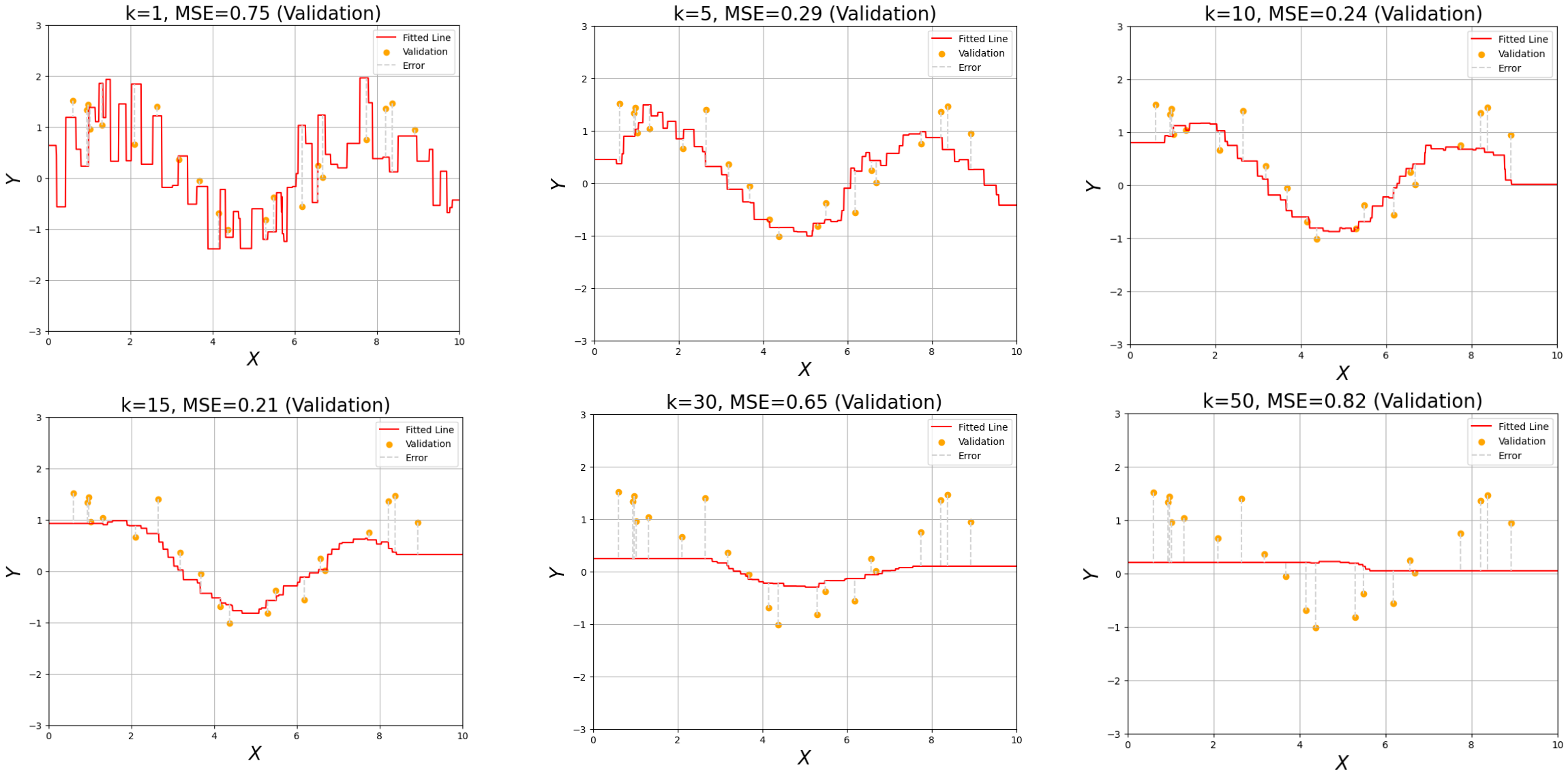
$$\text{MSE} \quad = \quad \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

where N is the total number of data points.

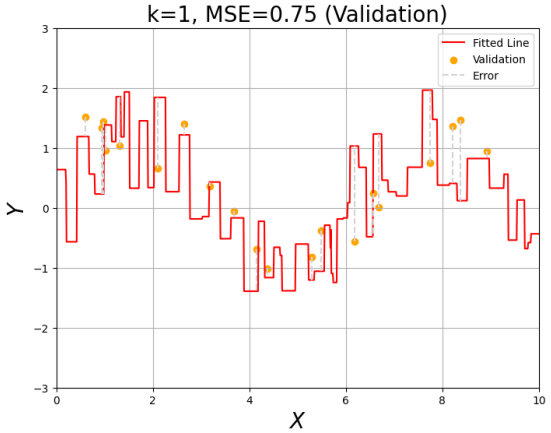
MyKNNRegressor (Revised)

```
from sklearn.base import BaseEstimator, RegressorMixin

class MyKNNRegressor(BaseEstimator, RegressorMixin):
    def __init__(self, k=3, distance_metric='euclidean'):
        self.k = k
        self.distance_metric = distance_metric

    def fit(self, X, y):
        # Store the training data
        self.X_train_ = X
        self.y_train_ = y
        return self

    def predict(self, X):
        # Custom predict logic for regression
        y_pred = []
        for x in X:
            # Calculate distances
            distances = pairwise_distances(self.X_train_, [x], metric=self.distance_metric).ravel()

            # Get indices of k nearest neighbors
            k_nearest_indices = np.argsort(distances)[:self.k]
            k_nearest_labels = self.y_train_[k_nearest_indices]

            # Compute the average of the nearest labels for regression
            predicted_value = np.mean(k_nearest_labels)
            y_pred.append(predicted_value)

        return np.array(y_pred)
```

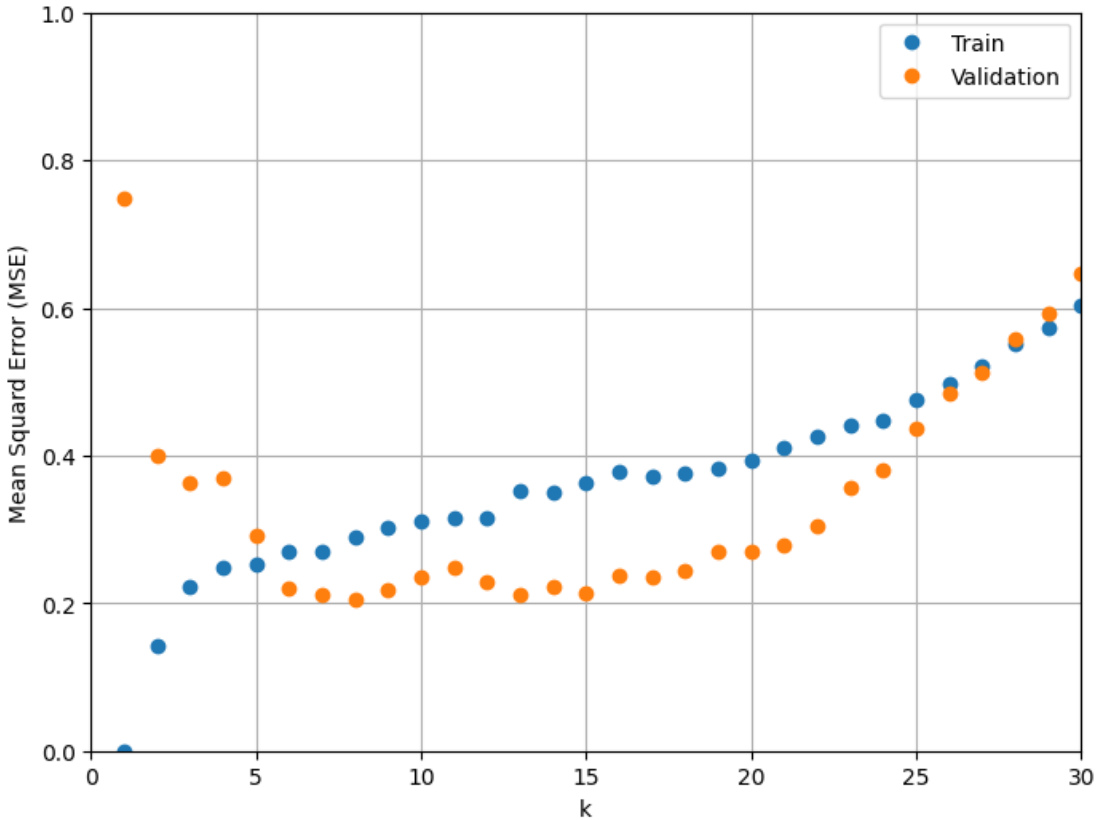
Model Complexity

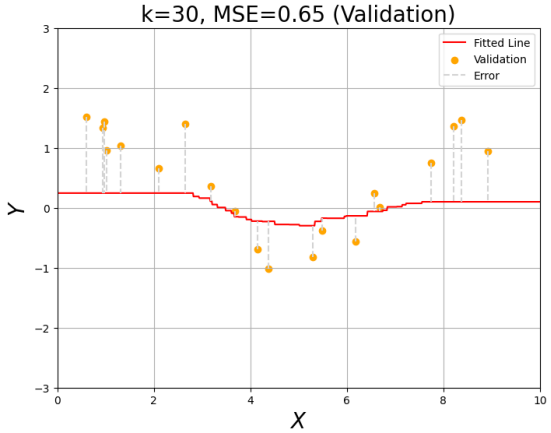


Bias-Variance Trade-Offs



- High Variance, Low Bias
- Complex Fitted Line
- Low Train MSE
- High Validation MSE

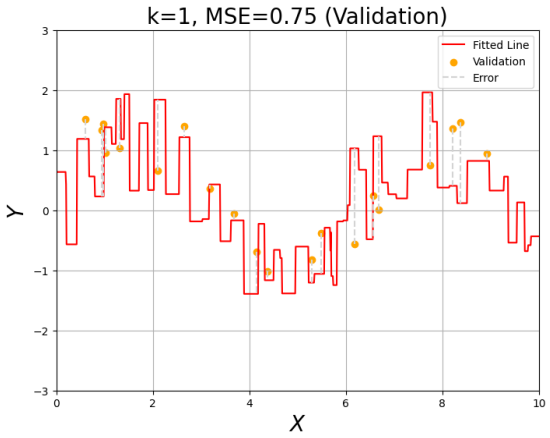




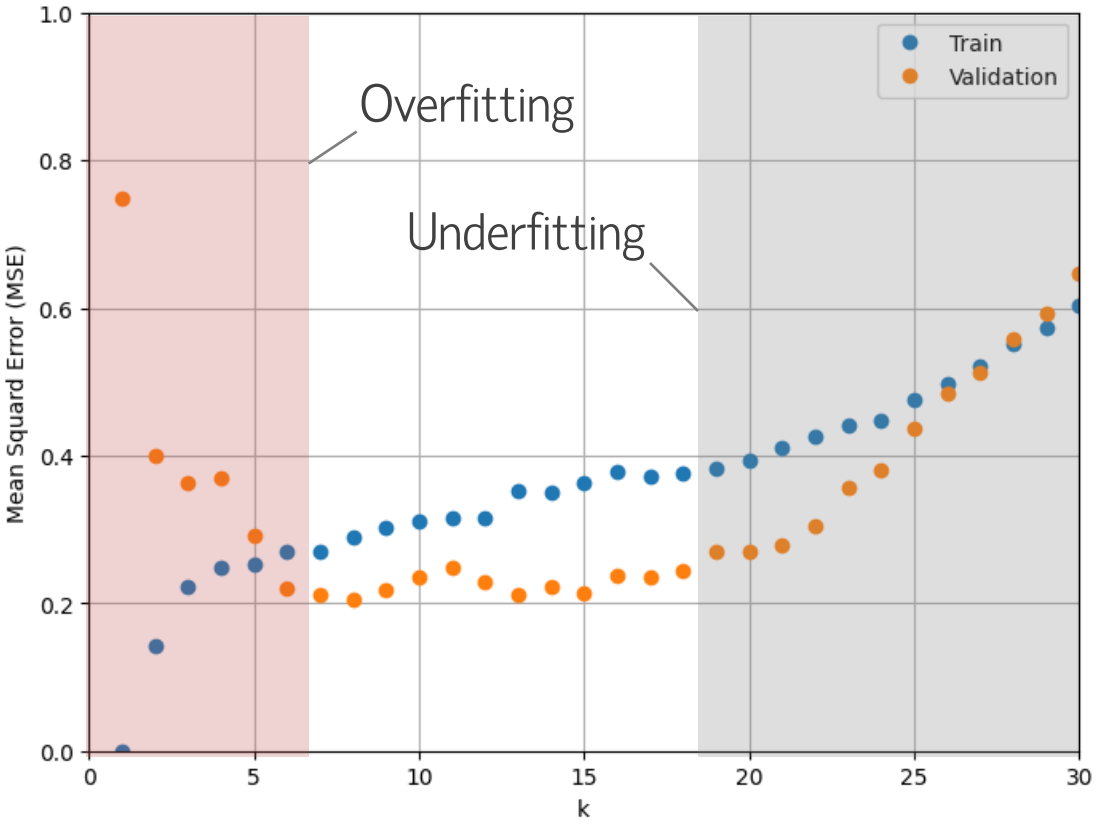
- High Bias, Low Variance
- Simple Fitted Line
- High Train MSE
- High Validation MSE

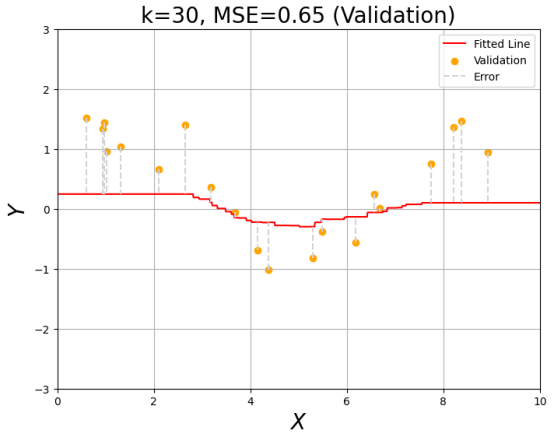
Overfitting vs Underfitting

- **Overfitting** occurs with a small k, where the model captures noise in the training data as if it were significant signal. This results in a model that performs well on the training data but poorly on validation data.
- **Underfitting** arises with a large k, leading to a model that is too generalized. It overlooks important patterns in the data, causing subpar performance on both the training and validation datasets as it fails to capture the underlying data structure.



- High Variance, Low Bias
- Complex Fitted Line
- Low Train MSE
- High Validation MSE

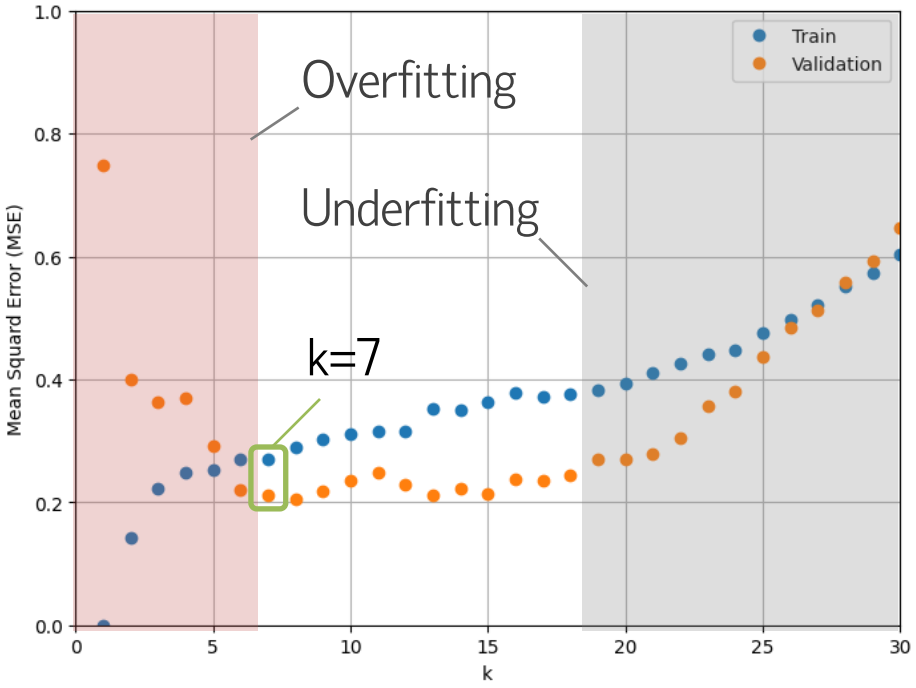


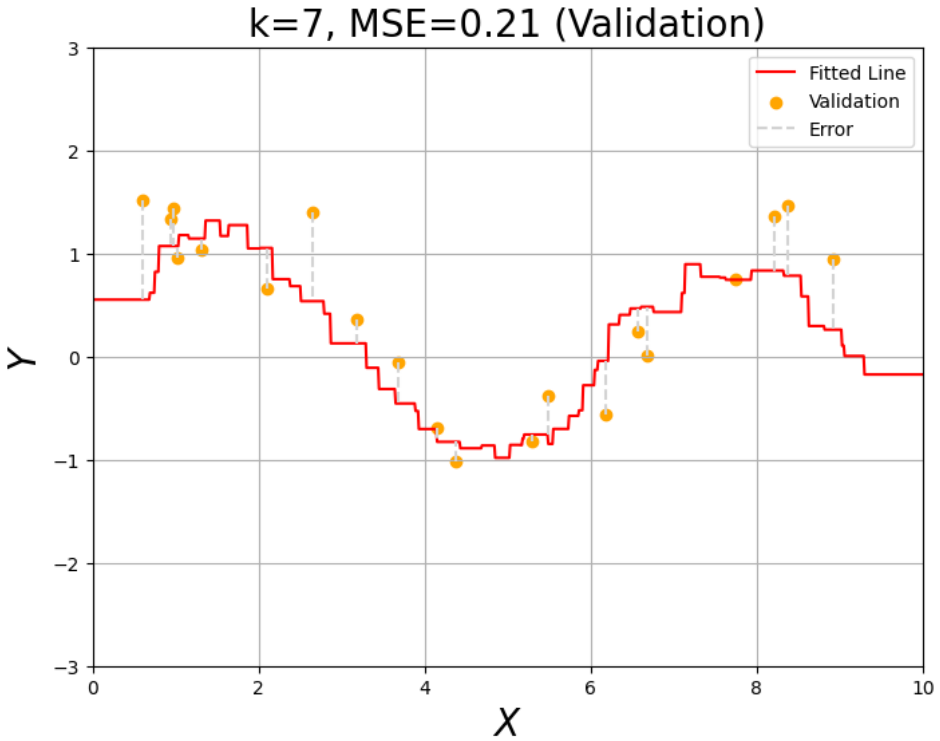


- High Bias, Low Variance
- Simple Fitted Line
- High Train MSE
- High Validation MSE

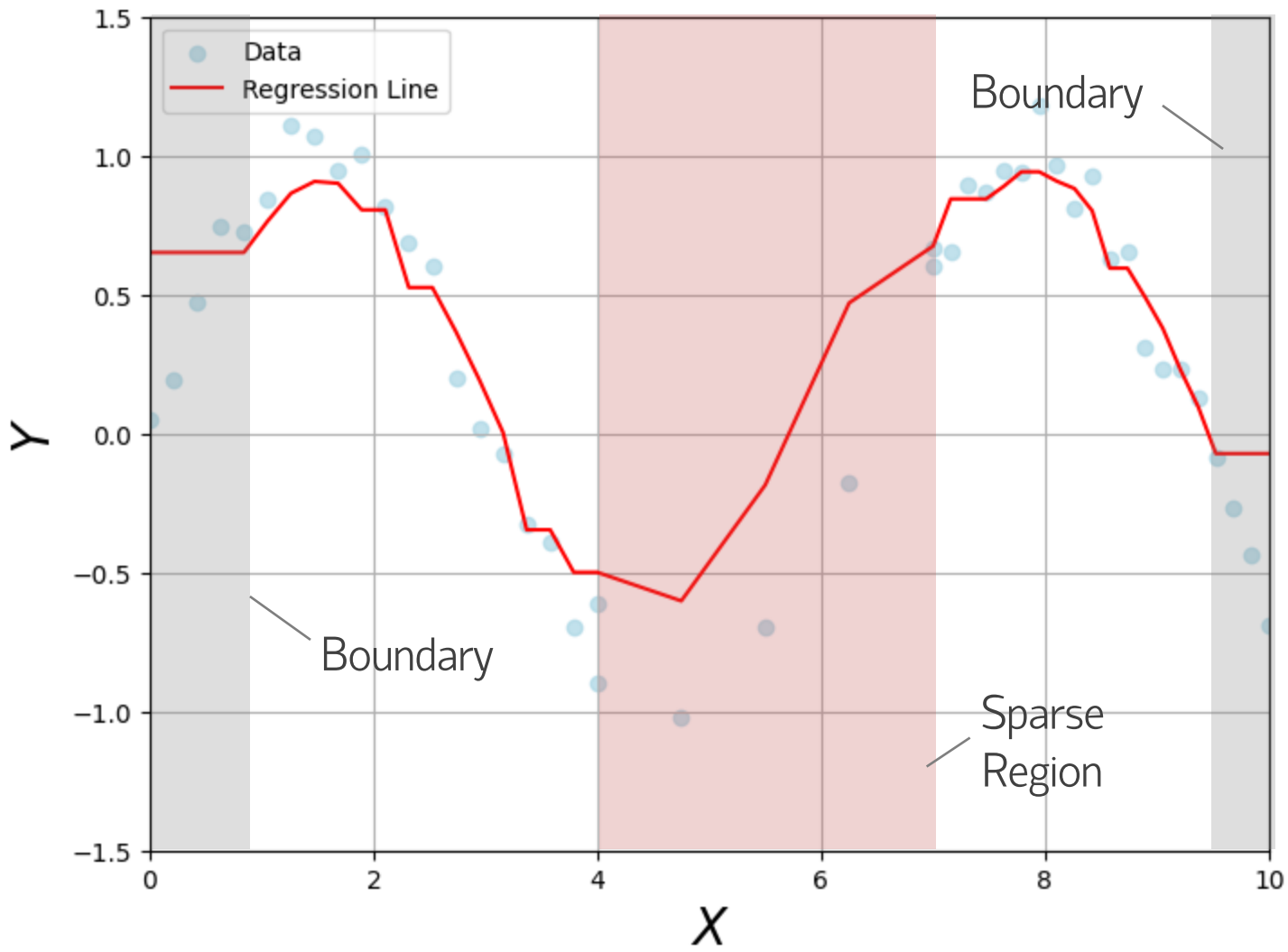
Optimal Model

- An optimal k-NN model is one where the choice of k strikes a balance between fitting the training data and generalizing to new data. The right k allows the model to smooth out noise and capture the underlying trends, leading to accurate predictions on both the training and unseen data, indicating it has avoided overfitting and underfitting.



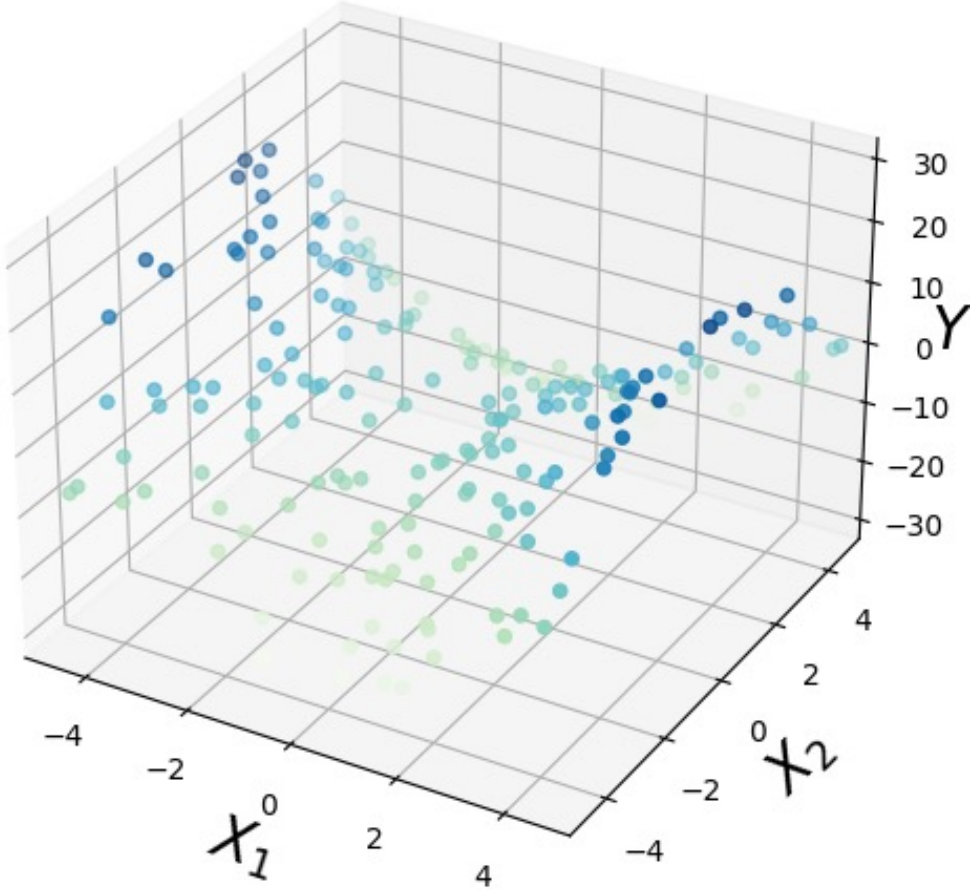


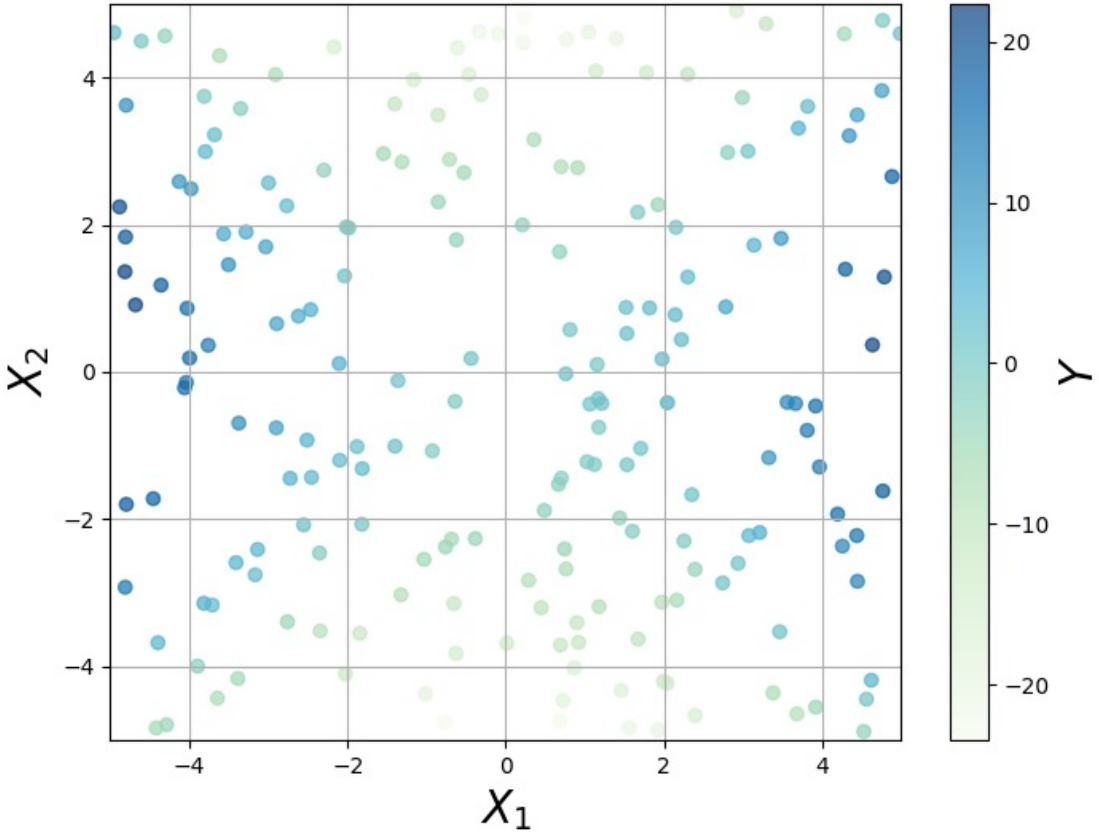
KNN in Practice



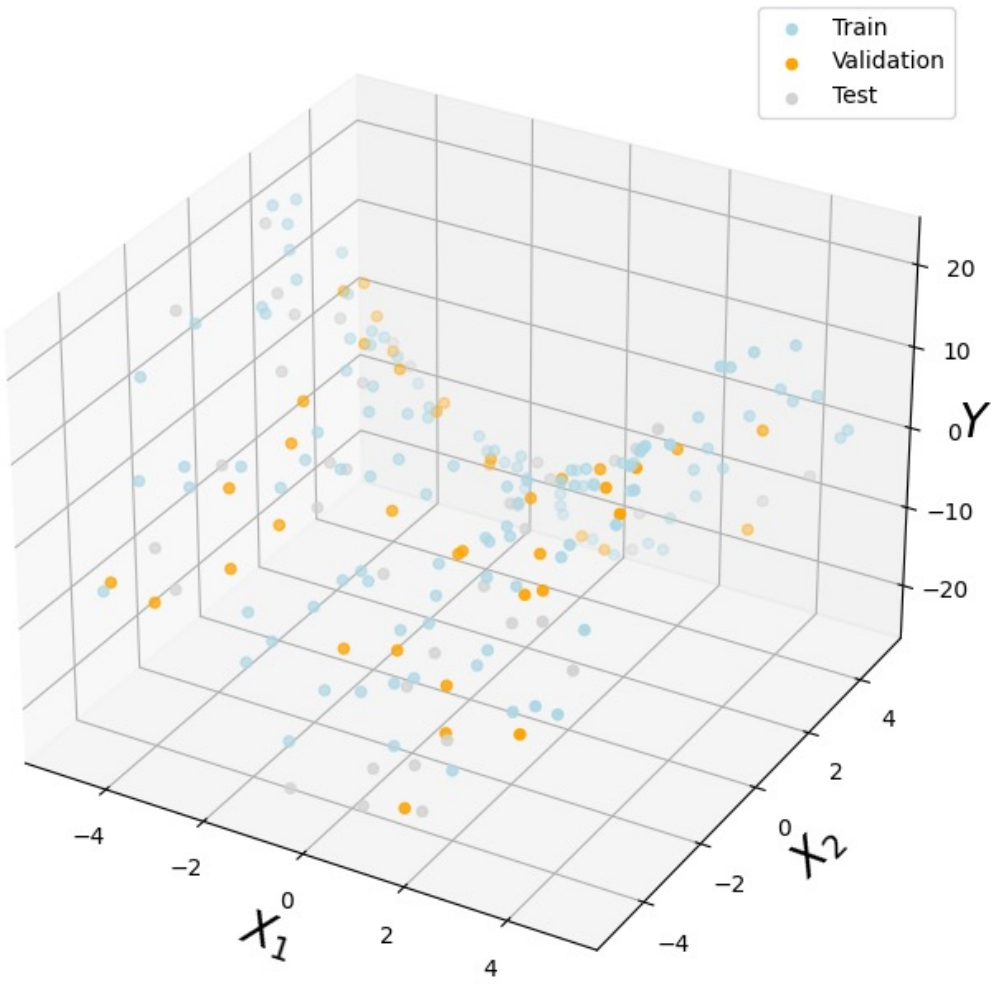
- The area, where there are high output variations among the neigbhours. is where the residues what observed and what predicted are likely to to be high. Hence, this is where regression errors are coming from.
- On the contrary, we can predict with low residue errors in the area where this is no (or little) variation in the target variable between the neighbouring points.

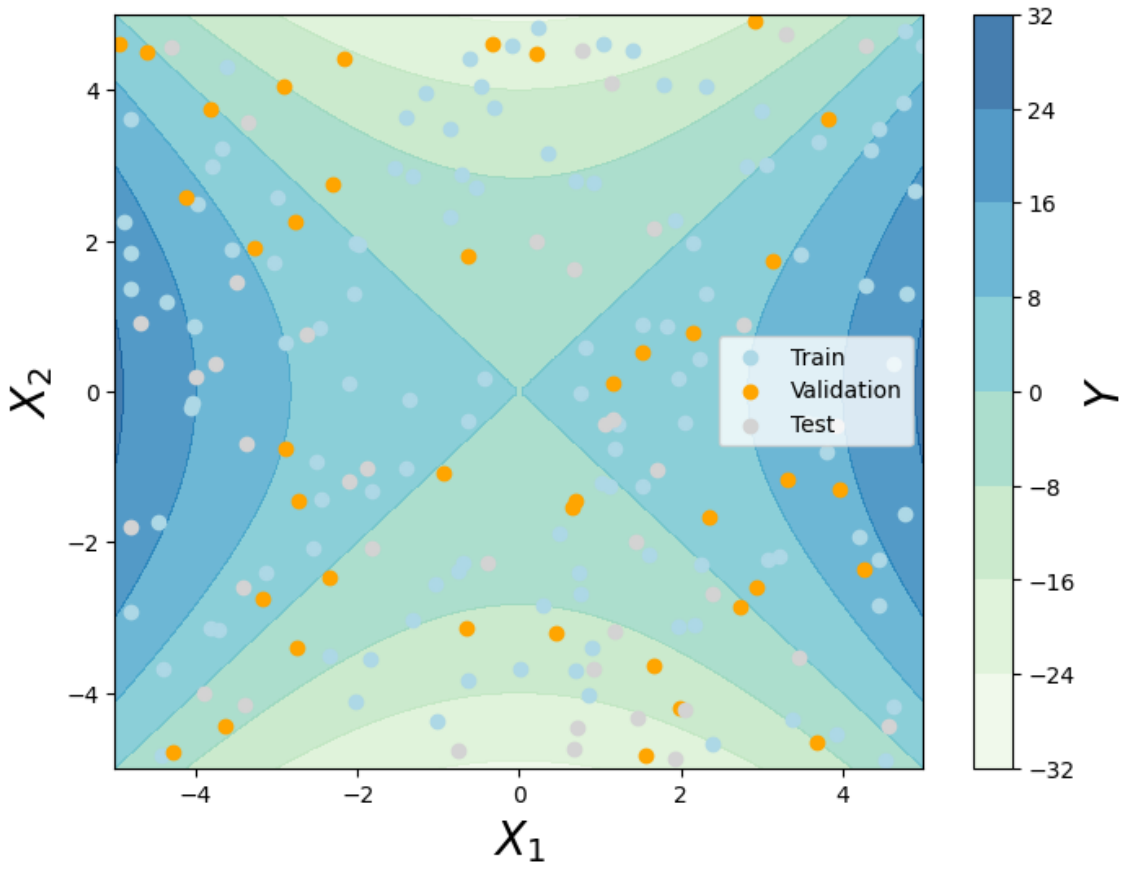
High Dimesional Dataset



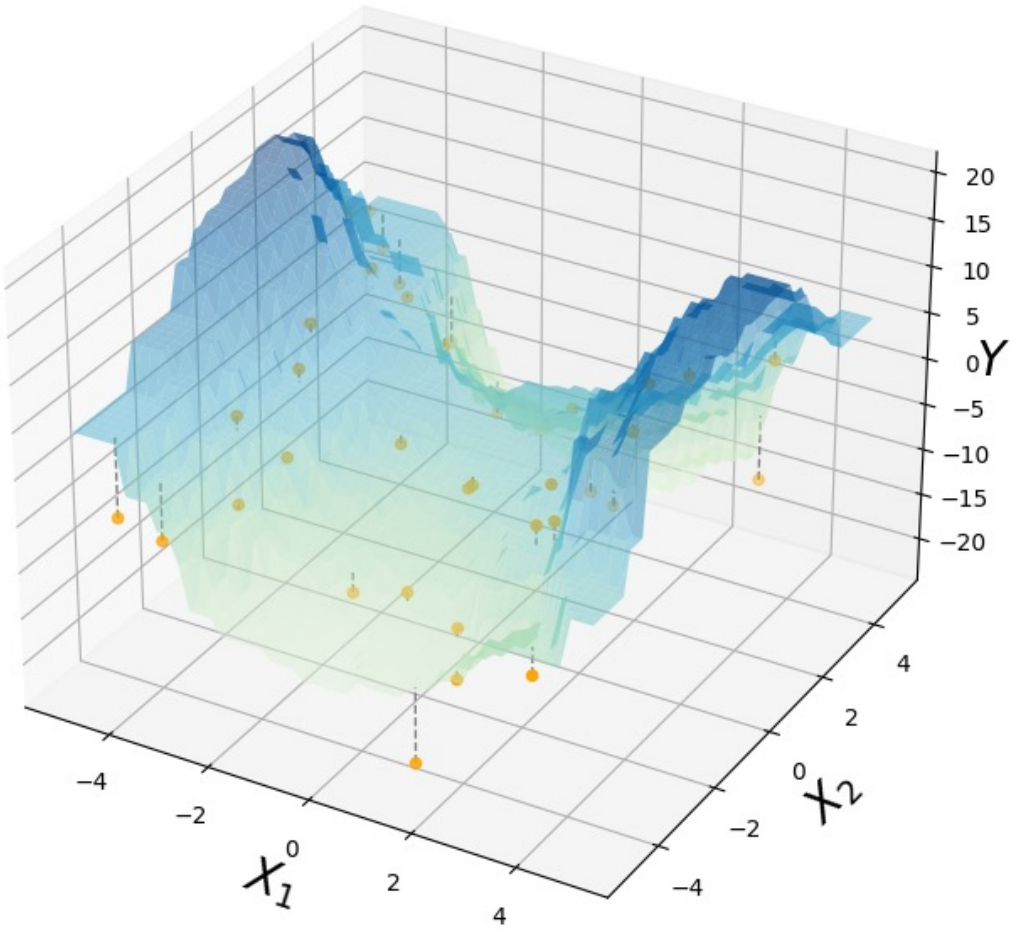


Train-Test Split





Regression Surface

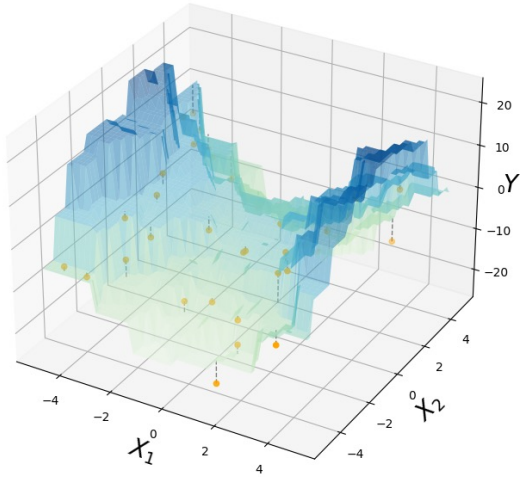


- It is similar to a regression line, but for higher dimensional data. The surface can be curved or flat, depending on the nature of the relationship.

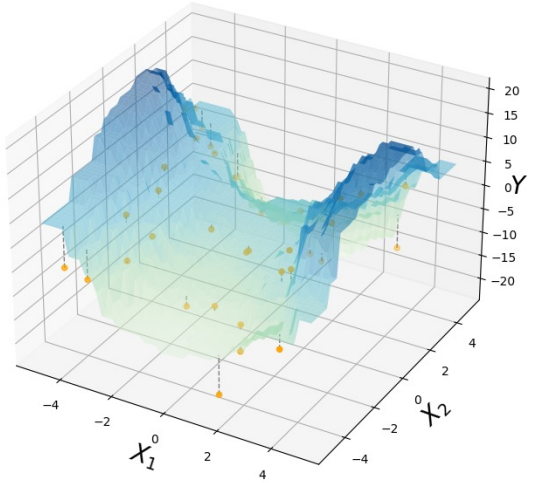
K = 5, MSE = 9.7 (Validation)

Model Complexity

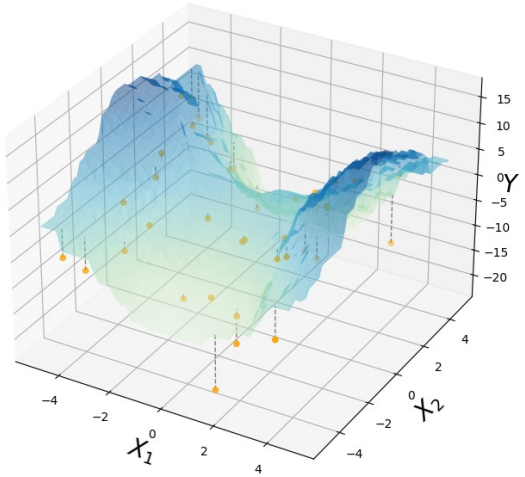
k=1, MSE=12.14 (Validation)



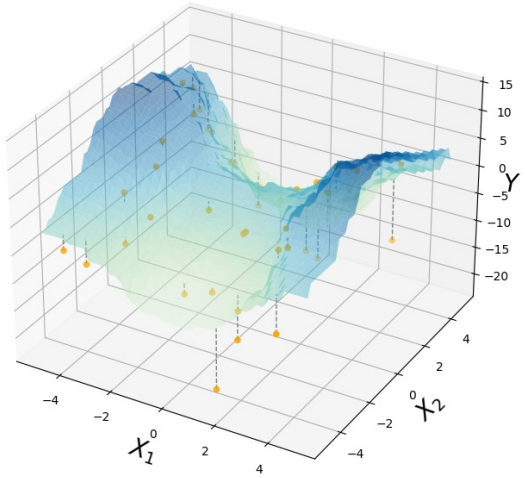
k=5, MSE=9.7 (Validation)



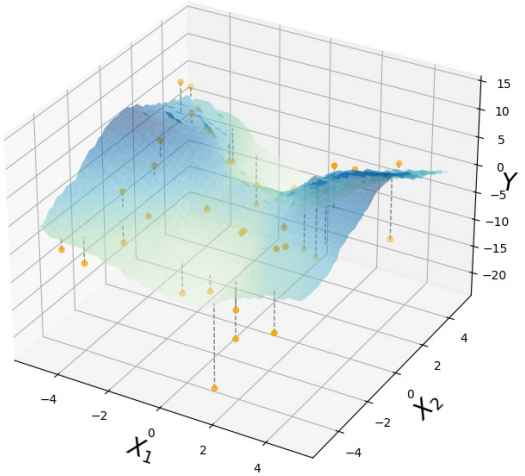
k=10, MSE=17.17 (Validation)



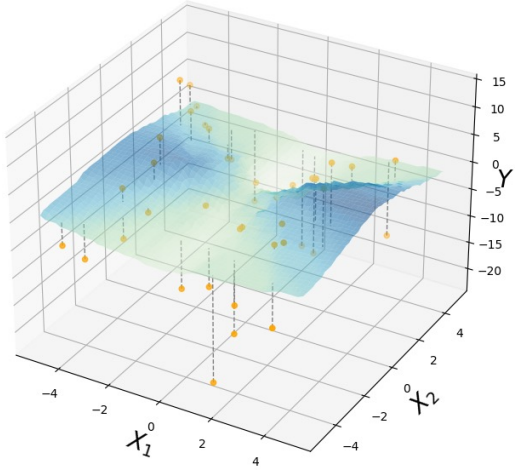
k=15, MSE=17.52 (Validation)



k=30, MSE=33.5 (Validation)

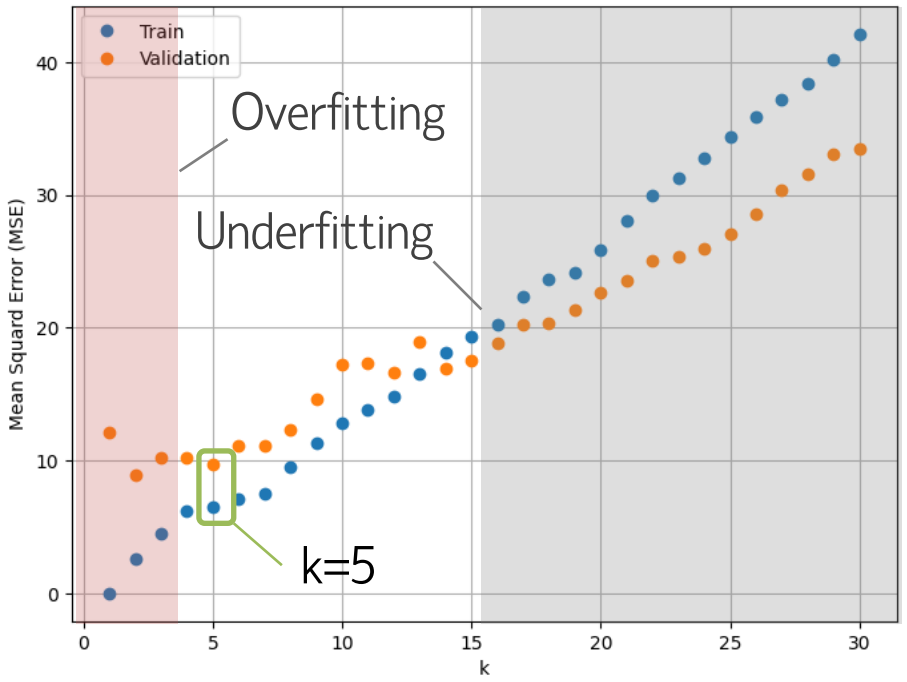


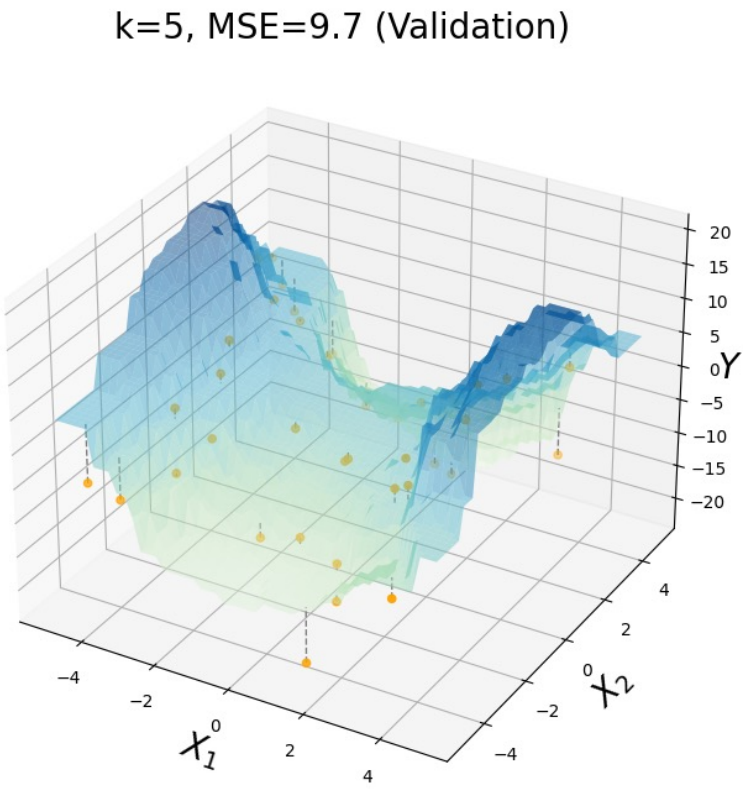
k=50, MSE=59.8 (Validation)



Optimal Model

- An optimal k-NN model is one where the choice of k strikes a balance between fitting the training data and generalizing to new data. The right k allows the model to smooth out noise and capture the underlying trends, leading to accurate predictions on both the training and unseen data, indicating it has avoided overfitting and underfitting.





Scaling Effects

A: (1.7 m, 60 kg), B: (1.8 m, 60 kg)

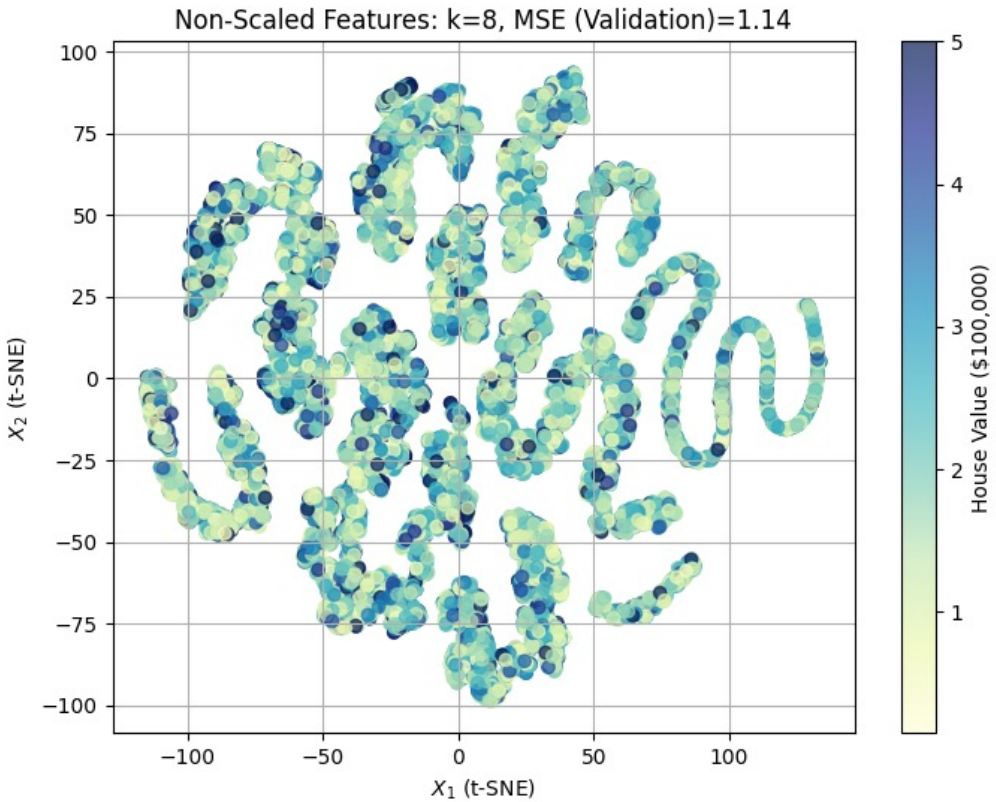
Distance(A, B) = 0.1

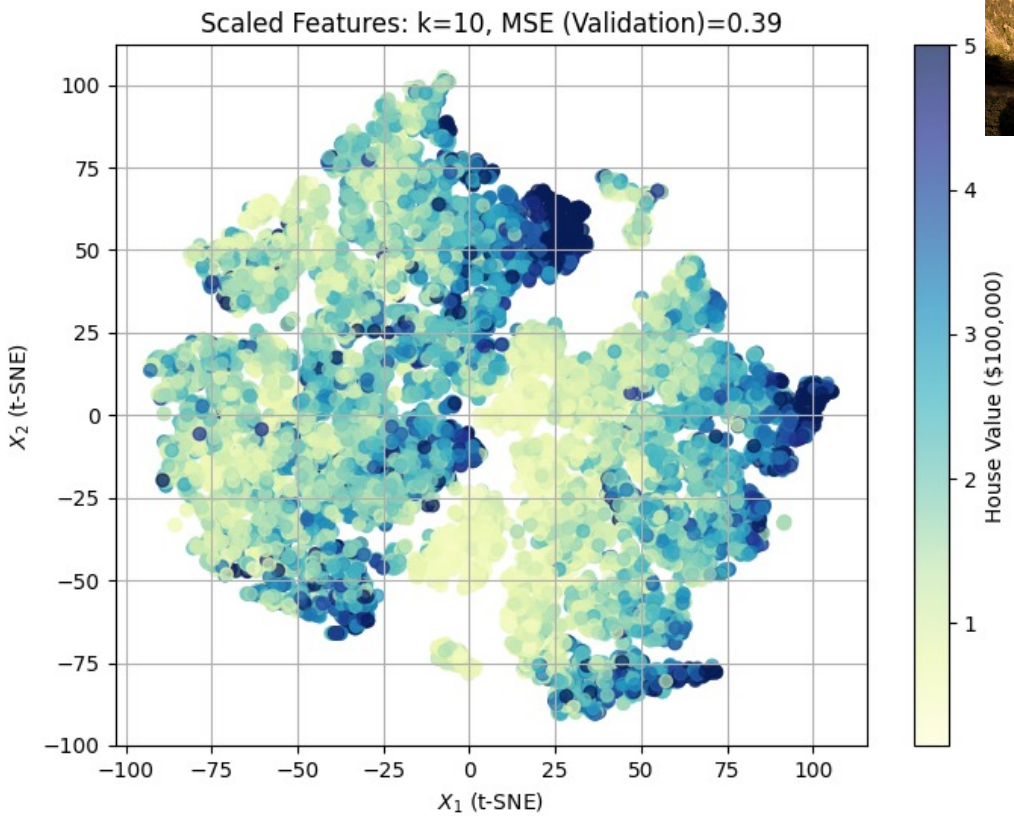
C: (1.7 m, 60 kg), D: (1.7 m, 70 kg)

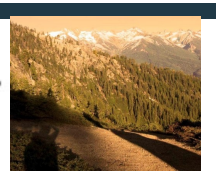
Distance(C, D) = 10

- Distance(C, D) is 100 times more than Distance(A, B). Eh?
- This is not right. The difference between A form B is very apparent, but not so for between C and B.
- We must ensure different measures and units are fairly contributing to the machine learning (ML) model.

California Housing Dataset: Better Predictions with Z-Score Features

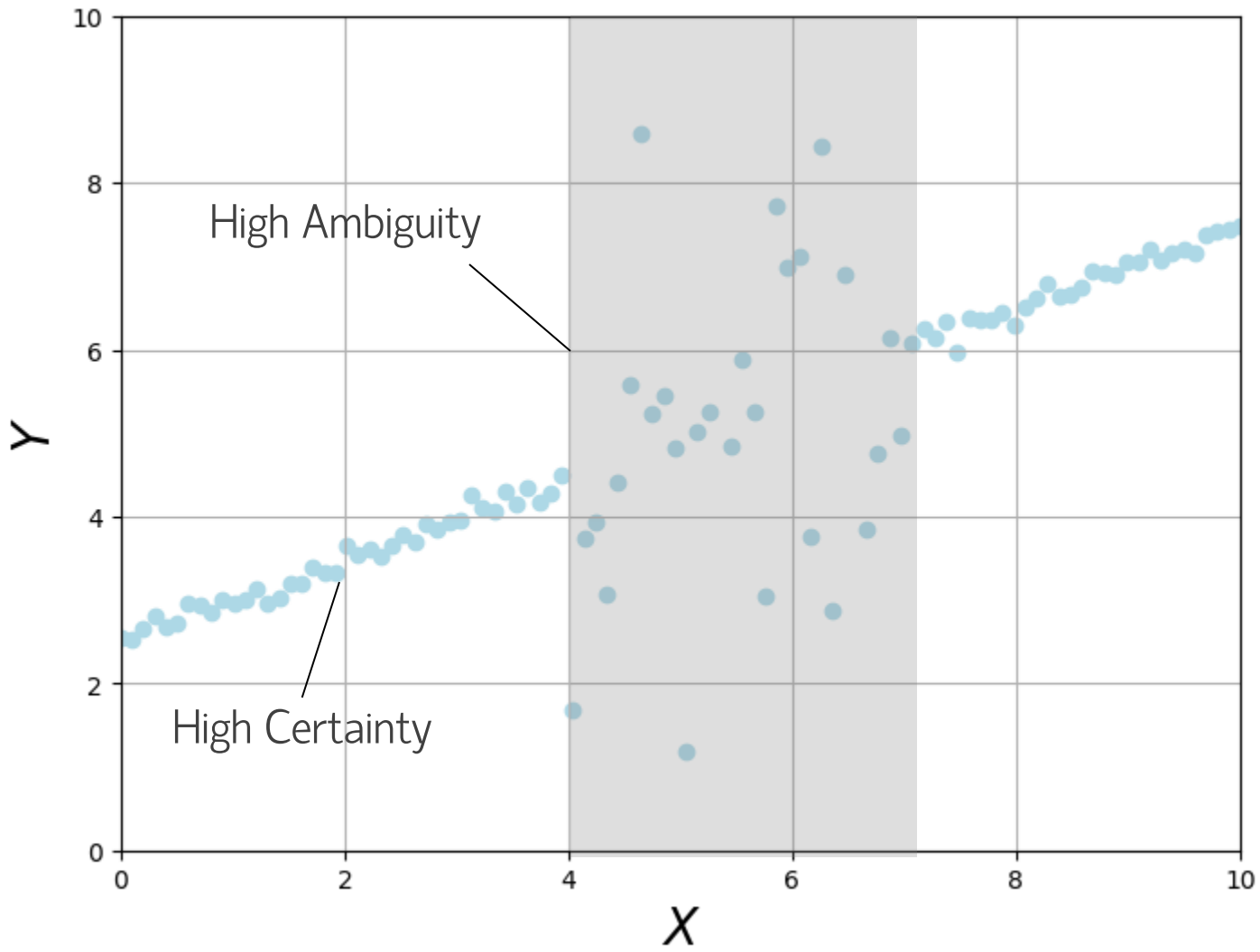






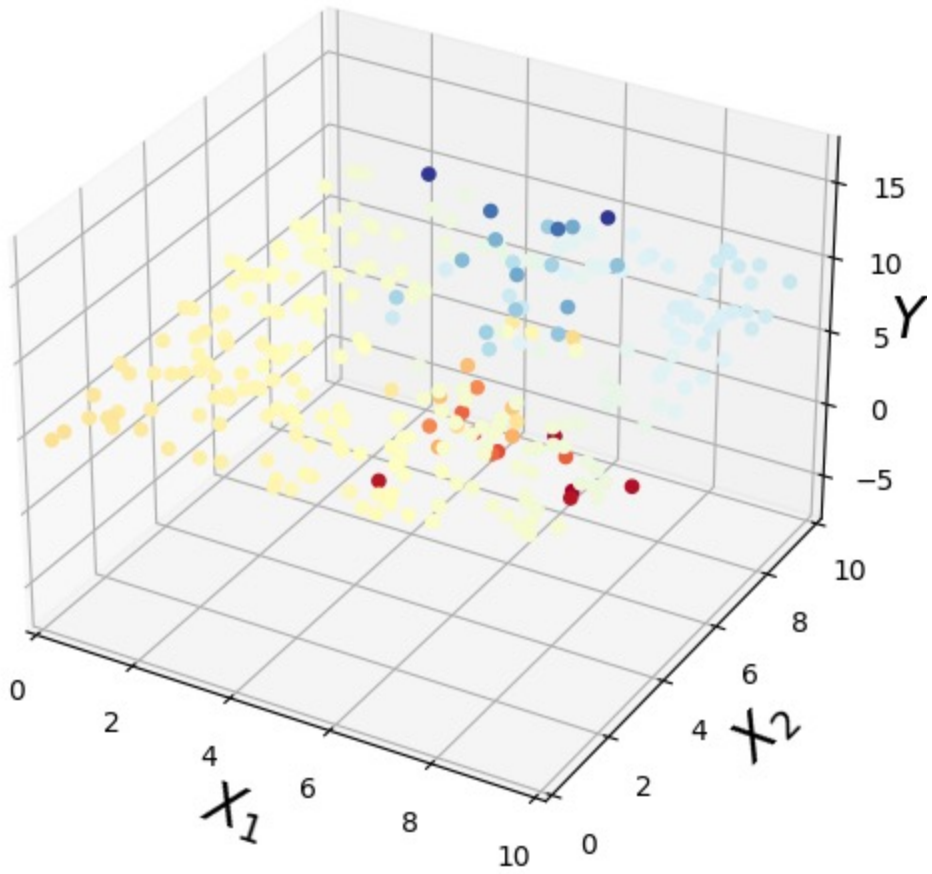
- California Housing Dataset: There are 8 features, which include median income, housing age, average rooms and bedrooms per household, population, average occupancy, and geographical coordinates (latitude and longitude) of the block group.
- Standardisation ensures features with different units and scales are contributing fairly into the ML model. This results in better separation between the classes.

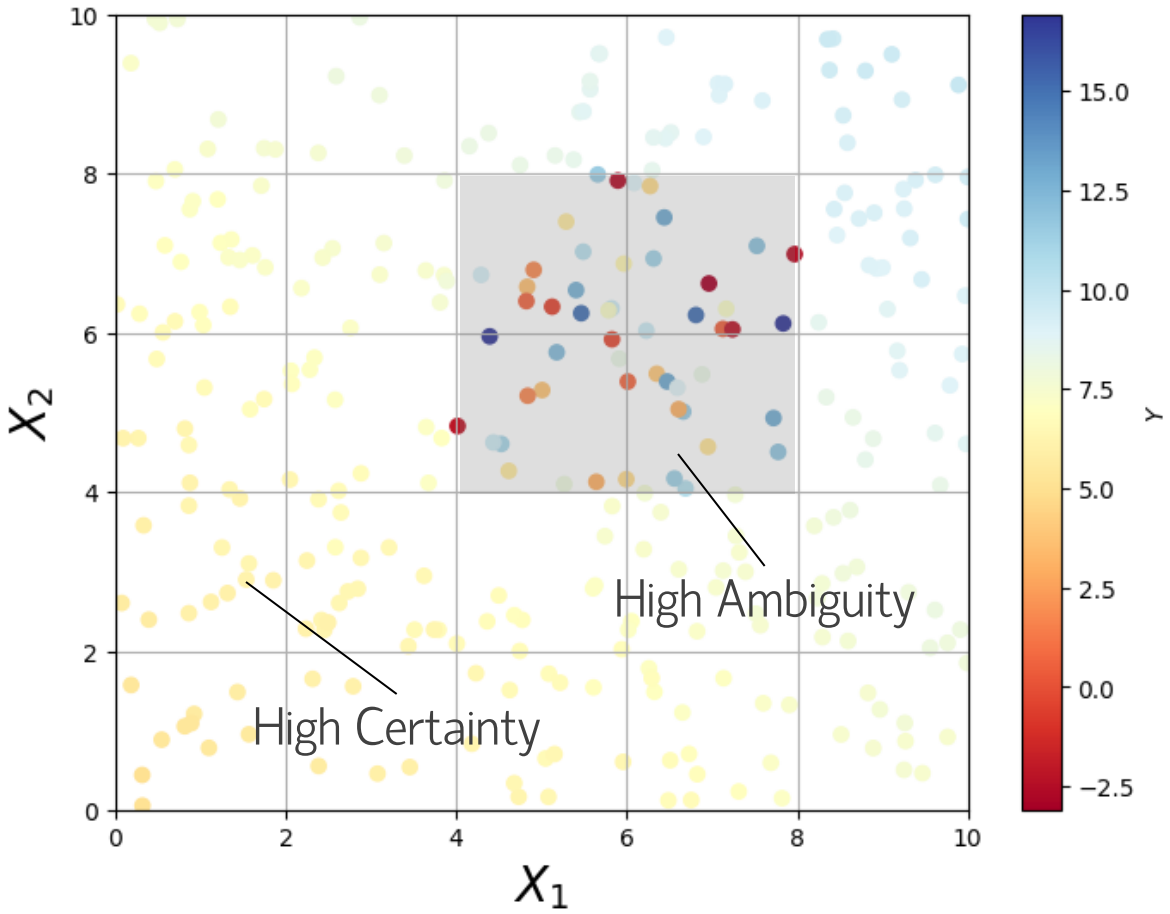
Where Do Errors Come From?



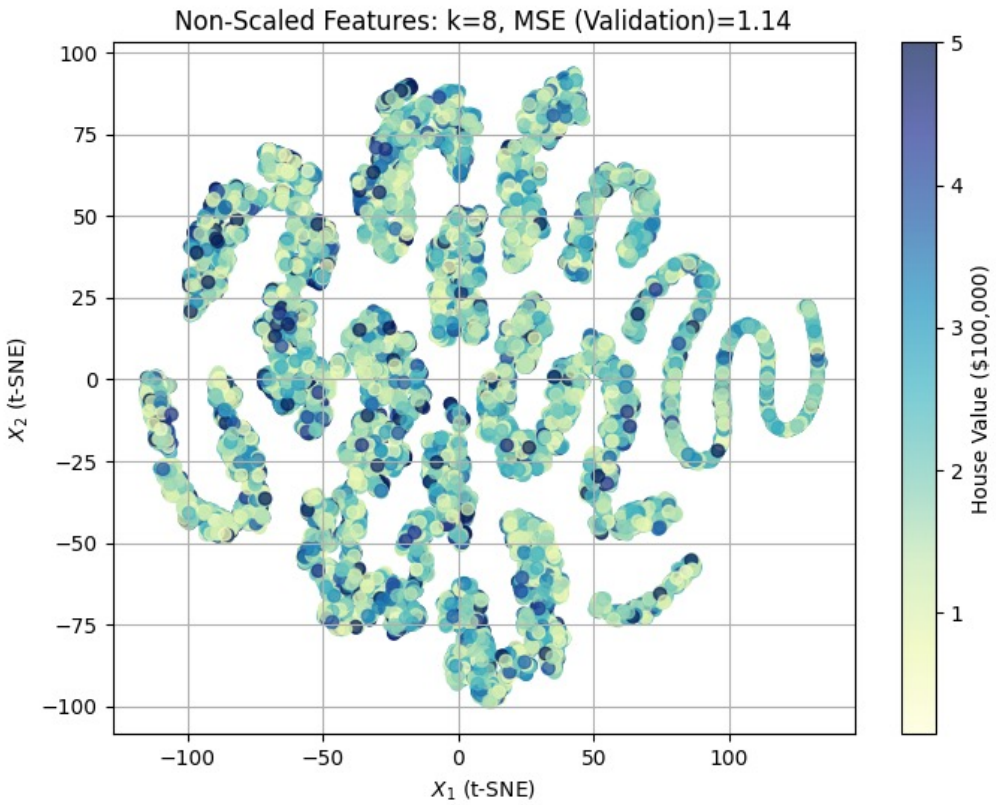
- The area, where there are high output variations among the neigbhours. is where the residues what observed and what predicted are likely to to be high. Hence, this is where regression errors are coming from.
- On the contrary, we can predict with low residue errors in the area where this is no (or little) variation in the target variable between the neighbouring points.

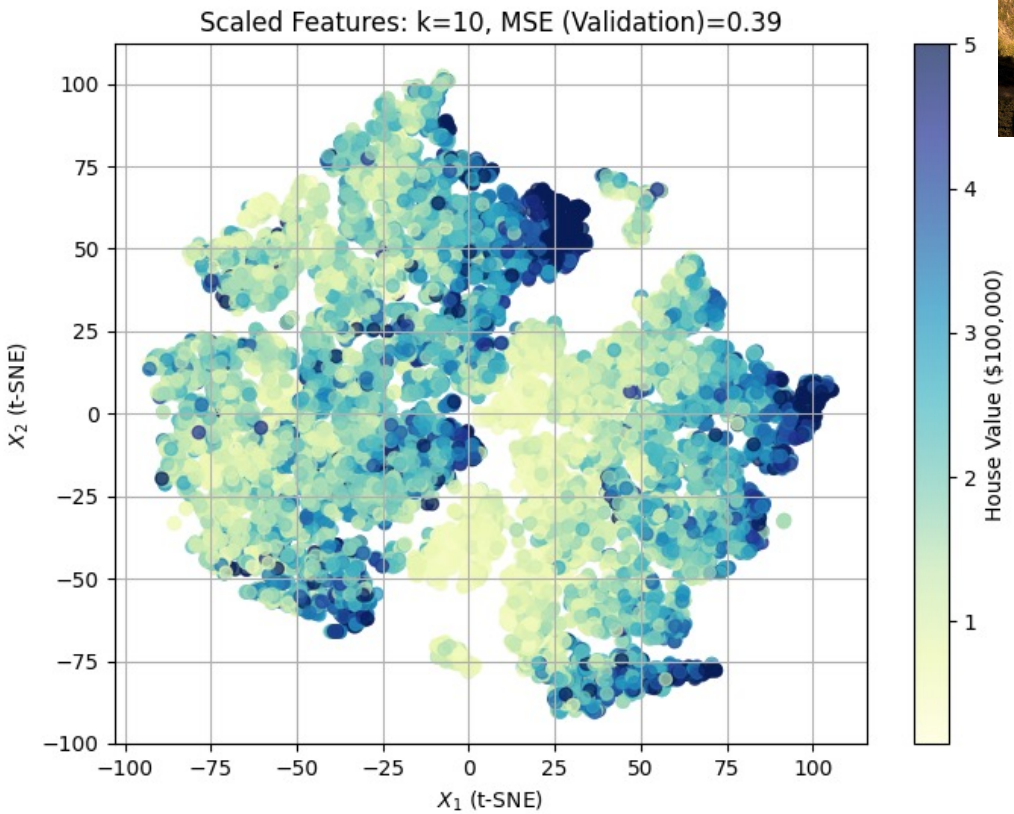
Where Do Errors Come From? (cont.)





California Housing Dataset: Z-Score Features (Revisit)





- For non-scaled features, there are high variations in term of price among the neighbouring data points. This will result in high prediction errors, hence a high MSE on the validation dataset.
- The house values are similar among the neighbouring data points with the standardised features. The change is also more gradual as the distance increases. We can predict with high certainty, and hence a low MSE.

Summary

- In regression problems, K-Nearest Neighbours (KNN) makes a prediction on query point by averaging the labels of its neighbouring data points. The notion of distance (or similarity) underpins how K-Nearest Neighbours (KNN) works. It defines nearness in the same manner as KNN classification.
- KNN models, or ML in general, are considered good if its prediction performances are consistent between train, validation and test datasets. We need to find a model that is neither overfitting nor underderfitting.
- Low K generally leads to a complex regression surface (or line), i.e. high variance (or low bias). In reverse, K will lead to a simple one, i.e. high bias (or low variance). We need to find K which will result in low validation mean squared error (MSE) and still maintaining low MSE on the training dataset.
- In practice, KNN performs poorly around the boundaries where k neighbouring points are unchanged. It has issues also in the regions where the data points are sparse. Nevertheless, KNN is much more reasonable fit in the presence of noise.
- In practice, we always scale our features before training a model. This often makes neighbouring data points to have comparable labels (or output values), and hence a better prediction performance.